Lab.1

1.

Program otwiera plik funkcją fopen("dane.txt", "w") w trybie zapisu ("w" – tworzy nowy plik lub czyści istniejący). Poprawność otwarcia sprawdzamy warunkiem if (plik == NULL) – jeśli zwrócono NULL, otwarcie się nie powiodło i program kończy się z kodem 1.

Pętla for zapisuje liczby od 1 do 10 za pomocą fprintf(plik, "%d\n", i) (każda w osobnej linii dzięki \n). Na końcu fclose(plik) zamyka plik i zapisuje dane na dysk.

```
root@debian:~# gcc zad1.c -o zad1
root@debian:~# ./zad1
Zapisano 10 liczb!
root@debian:~# cat dane.txt
1
2
3
4
5
6
7
8
9
10
root@debian:~#
```

```
  GNU nano 7.2
#include <stdio.h>
int main()
{
FILE *plik = fopen("dane.txt", "w");
if (plik == NULL)
{
printf("Blad!\n");
return 1;
}
for (int i = 1; i <=10; i++)
fprintf(plik, "%d\n", i);
fclose(plik);
printf("Zapisano 10 liczb!\n");
return 0;
}
```

2.

Plik otwieramy w trybie odczytu fopen("dane.txt", "r"). Jeśli plik nie istnieje, fopen zwraca NULL wtedy wypisujemy komunikat i kończymy program (to jest właśnie obsługa braku pliku).

Zawartość czytamy w pętli funkcją fgets(linia, sizeof(linia), plik), która wczytuje po jednej linii do bufora, aż do końca pliku (gdy zwróci NULL). Każdą linię od razu wypisujemy przez printf.

```
  GNU nano 7.2
#include <stdio.h>
int main()
{
FILE *plik = fopen("dane.txt", "r");
if (plik == NULL)
{
printf("Plik ni istnieje!\n");
return 1;
}
char linia[256];
while (fgets(linia, sizeof(linia), plik) != NULL)
{
printf("%s", linia);
}
fclose(plik);
return 0;
}
```

```
root@debian:~# gcc zad2.c -o zad2
root@debian:~# ./zad2
1
2
3
4
5
6
7
8
9
10
root@debian:~#
```

3.

Program czyta plik znak po znaku w pętli while ((c = fgetc(plik)) != EOF). Funkcja fgetc zwraca kolejny znak lub stałą EOF na końcu pliku.

Dla każdego znaku zwiększamy licznik znaków. Jeśli znak to '\n', zwiększamy licznik linii; jeśli to ' ' (spacja) licznik spacji. Na końcu wypisujemy trzy wyniki.

Uwaga: zmienną c deklarujemy jako int (a nie char), bo EOF jest wartością całkowitą i nie zmieściłaby się poprawnie w typie char.

```
  GNU nano 7.2
#include <stdio.h>

int main()
{
FILE *plik = fopen("dane.txt", "r");
if (plik == NULL)
{
printf("Blad!\n");
return 1;
}
int znaki = 0, linie = 0, spacje = 0, c;
while ((c = fgetc(plik)) != EOF)
{
znaki++;
if (c == '\n') linie++;
if (c == ' ') spacje++;
}
fclose(plik);
printf("Liczba znakow: %d\n", znaki);
printf("Liczba linii: %d\n", linie);
printf("Liczba spacji: %d\n", spacje);
return 0;
}
```

```
root@debian:~# gcc zad3.c -o zad3
root@debian:~# ./zad3
Liczba znakow: 21
Liczba linii: 10
Liczba spacji: 0
root@debian:~#
```

4.

Kluczowy jest tryb otwarcia fopen("dane.txt", "a") ("a" = append, czyli **dopisywanie**). W tym trybie nowe dane są zawsze dodawane na końcu pliku, a istniejąca zawartość nie jest kasowana (w przeciwieństwie do trybu "w").

Linię od użytkownika pobieramy przez fgets(linia, sizeof(linia), stdin), zapisujemy do pliku przez fprintf, a następnie zamykamy plik funkcją fclose.

```
  GNU nano 7.2
#include <stdio.h>

int main()
{
FILE *plik = fopen("dane.txt", "a");
if (plik == NULL)
{
printf("Blad!\n");
return 1;
}
char linia[256];
printf("Podaj linie tekstu: ");
fgets(linia, sizeof(linia), stdin);
fprintf(plik, "%s", linia);
fclose(plik);
printf("Dopisano linie do pliku!\n");
return 0;
}
```

```
root@debian:~# gcc zad4.c -o zad4
root@debian:~# ./zad4
Podaj linie tekstu: moj tekst
Dopisano linie do pliku!
root@debian:~# cat dane.txt
1
2
3
4
5
6
7
8
9
10
moj tekst
root@debian:~#
```

5.

Plik binarny otwieramy w trybie "wb" (write binary). Liczby zapisujemy funkcją fwrite(&liczba, sizeof(int), 1, plik) – zapisuje ona surowe bajty liczby, a nie tekst. W przykładzie zapisujemy 20 liczb: 0, 10, 20, … 190.

Następnie otwieramy plik w trybie "rb" i używamy fseek(plik, pozycja * sizeof(int), SEEK_SET), aby ustawić wskaźnik pliku na konkretnej liczbie (licząc od początku – SEEK_SET). Funkcja fread odczytuje tę jedną liczbę. Dla pozycji 5 wynik to 50.

```
int main()
{
FILE *plik = fopen("liczby.bin", "wb");
if (plik == NULL)
{
printf("Blad!\n");
return 1;
}
int i;
for (i = 0; i < 20; i++)
{
int l = i * 10;
fwrite(&l, sizeof(int), 1, plik);
}
fclose(plik);
plik = fopen("liczby.bin", "rb");
if (plik == NULL)
{
printf("Blad!\n");
return 1;
}
int poz = 5;
fseek(plik, poz * sizeof(int), SEEK_SET);
int wynik;
fread(&wynik, sizeof(int), 1, plik);
printf("Liczba na pozycji %d: %d\n", poz, wynik);
fclose(plik);
return 0;
}
```

```
root@debian:~# gcc zad5.c -o zad5
root@debian:~# ./zad5
Liczba na pozycji 5: 50
root@debian:~# _
```

6.

Definiujemy strukturę Rekord z polami int id i float wartosc. Całą tablicę 5 struktur zapisujemy jednym wywołaniem fwrite(dane, sizeof(struct Rekord), 5, plik) – zaletą zapisu binarnego jest to, że cała struktura trafia do pliku „w jednym kawałku”.

Przy odczycie otwieramy plik w trybie "rb" i czytamy w pętli while (fread(&odczyt, sizeof(struct Rekord), 1, plik) == 1) – pętla działa, dopóki uda się odczytać kompletną strukturę. Każdy rekord wypisujemy w formacie „ID, Wartosc”.

```
  GNU nano 7.2                                                    zad6
#include <stdio.h>

struct Rekord
{
int id;
float wartosc;
};

int main()
{
struct Rekord dane[5] = {{1,10.5}, {2,20.0}, {3,33.3}, {4,4.75}, {5,100.0}};
FILE *plik = fopen("rekordy.bin", "wb");
if (plik == NULL) { printf("blad!\n"); return 1; }
fwrite(dane, sizeof(struct Rekord), 5, plik);
fclose(plik);
plik = fopen("rekordy.bin", "rb");
if (plik == NULL) { printf("Blad!\n"); return 1; }
struct Rekord r;
while (fread(&r, sizeof(struct Rekord), 1, plik) == 1)
{
printf("ID: %d, Wartosc: %.2f\n", r.id, r.wartosc);
}
fclose(plik);
return 0;
}
```

```
root@debian:~# ./zad6
ID: 1, Wartosc: 10.50
ID: 2, Wartosc: 20.00
ID: 3, Wartosc: 33.30
ID: 4, Wartosc: 4.75
ID: 5, Wartosc: 100.00
root@debian:~#
```

7.

Tutaj używamy niskopoziomowego, systemowego wywołania open() (z nagłówka <fcntl.h>), a nie fopen. Flagi O_CREAT | O_WRONLY oznaczają: utwórz plik, jeśli nie istnieje, i otwórz go tylko do zapisu. Trzeci argument 0644 to uprawnienia w systemie ósemkowym.

Znaczenie 0644 (rw-r--r--):

Cyfra / kto	Uprawnienia
6 – właściciel	odczyt + zapis (rw-)
4 – grupa	tylko odczyt (r--)
4 – inni	tylko odczyt (r--)

Czy mają sens? Tak. 0644 to standardowe i bezpieczne uprawnienia dla zwykłych plików: właściciel może je modyfikować, a pozostali użytkownicy tylko czytać. Dzięki temu plik nie jest przypadkowo zapisywalny przez innych. Po sprawdzeniu poleceniem ls -l widać -rw-r—r--.

```
 GNU nano 7.2
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
int fd = open("nowy.txt", O_CREAT | O_WRONLY, 0644);
if (fd == -1)
{
printf("BLad!\n");
return 1;
}
printf("Plik utworzony z uprawieniami 0644\n");
close(fd);
return 0;
}
```

```
root@debian:~# gcc zad7.c -o zad7
root@debian:~# ./zad7
Plik utworzony z uprawieniami 0644
root@debian:~# ls -l nowy.txt
-rw-r--r-- 1 root root 0 Jun  3 18:49 nowy.txt
root@debian:~# _
```

8.

Rozbudowałem program z Zadania 2 o pełną obsługę błędów. Zamiast zwykłego printf używam funkcji perror, która wypisuje podany komunikat oraz automatycznie dopisuje systemowy opis błędu (na podstawie zmiennej errno), np. „No such file or directory”.

Sprawdzane są trzy operacje: otwarcie (fopen), błąd odczytu (ferror) oraz zamknięcie (fclose). Każdy błąd kończy program innym kodem wyjścia (exit(1), 2, 3) – dzięki temu można rozpoznać, na którym etapie wystąpił problem.

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main()
{
FILE *plik = fopen("dane.txt", "r");
if (plik == NULL)
{
perror("Blad otwarcia pliku");
exit(1);
}
char linia[256];
while (fgets(linia, sizeof(linia), plik) != NULL)
{
printf("%s", linia);
}
if (ferror(plik))
{
perror("BLad odczytu");
fclose(plik);
exit(2);
}
if (fclose(plik) != 0)
{
perror("Blad zamkniecia");
exit(3);
}
return 0;
}
```

Na zrzucie warto pokazać dwa przypadki: poprawne działanie oraz uruchomienie przy braku pliku (po rm dane.txt), gdzie perror wypisuje błąd.

```
root@debian:~# gcc zad8.c -o zad8
root@debian:~# ./zad
-bash: ./zad: No such file or directory
root@debian:~# ./zad8
1
2
3
4
5
6
7
8
9
10
moj tekst
root@debian:~# rm dane.txt
root@debian:~# ./zad8
Blad otwarcia pliku: No such file or directory
root@debian:~#
```

9.

Program otwiera dwa pliki: źródłowy (log.txt) w trybie odczytu i docelowy (kopia.txt) w trybie zapisu. Oba otwarcia są sprawdzane pod kątem błędu (perror).

Plik czytamy linia po linii (fgets). Dla każdej linii sprawdzamy funkcją strstr(linia, "error"), czy zawiera szukany ciąg. Funkcja strstr zwraca wskaźnik na znalezione miejsce lub NULL. Jeśli ciąg występuje, linię zapisujemy do pliku kopii przez fputs. Na końcu wypisujemy, ile linii skopiowano.

```
  GNU nano 7.2
#include <stdio.h>
#include <string.h>

int main()
{
FILE *we = fopen("log.txt", "r");
if (we == NULL)
{
perror("BLad otwarcia pliku");
return 1;
}
FILE *wy = fopen("kopia.txt", "w");
if (wy == NULL)
{
perror("Blad utworzenia pliku");
fclose(we);
return 1;
}
char linia[256];
int liczba = 0;
while (fgets(linia, sizeof(linia), we) != NULL)
{
if (strstr(linia, "error") != NULL)
{
fputs(linia, wy);
liczba++;
}
}
fclose(we);
fclose(wy);
printf("Skopiowano %d linii z error\n", liczba);
return 0;
}
```

```
root@debian:~# ./zad1
Zapisano 10 liczb!
root@debian:~# ./zad9
Skopiowano 2 linii z error
root@debian:~# cat kopia.txt
error:brak pliku
error: brak pamieci
root@debian:~#
```